



Bases de datos I

Wilmer Hernán Ortega Erazo

Facultad de ingeniería

Objetivos de Aprendizaje

- ✓ Describir las características generales del SQL.
- ✓ Identificar los tipos de construcción en el lenguaje SQL.
- ✓ Crear una base de datos, las entidades y definir sus atributos y restricciones básicas.
- ✓ Transformar una consulta de álgebra relacional en una consulta SQL.
- ✓ Implementar solución a consultas complejas mediante el uso de consultas SQL compuestas y subconsultas SQL.

El Lenguaje de Consulta SQL

*SQL: desarrollado en los **laboratorios de investigación de IBM** en San José, California durante los años 70 por **Donald D. Chamberlin**. El nombre inicial fue **SEQUEL** (Structures English QUery Language), para 1980 el producto se rebautizó como **SQL**.*

*Actualmente es el lenguaje de manejo de datos relacionales más importante. Ha recibido la aceptación del **American National Standards Institute (ANSI)** como lenguaje para el manejo de las bases de datos, y es el lenguaje de acceso a los datos que se aplica en muchos DBMS comerciales (DB2, SQL7DS, ORACLE, INFORMIX, SYBASE, POSTGRES, MYSQL, SQL Server, Paradox, Microsoft ACCESS, etc.).*

Las ordenes de SQL se pueden usar interactivamente como lenguaje de consulta o se pueden insertar en programas de aplicación.

El Lenguaje de Consulta SQL

SQL es un lenguaje que transforma una o más relaciones como entrada, en una relación de salida. **El resultado de cada consulta SQL es una relación.**

Las ordenes de SQL se pueden usar interactivamente como lenguaje de consulta o se pueden insertar en programas de aplicación.

El nombre SQL sugiere ser un lenguaje de consulta, incluye definición de datos, actualización de la base de datos, definición de vistas, control de seguridad y facilidades de consulta.

La version original de SQL, fue diseñada para manipular y recuperar datos almacenados en el Sistema de base de datos original de IBM, conocido como System R.

El Lenguaje de Consulta SQL

Sea la siguiente base de datos:

Empleado

<u>codigo</u>	nombre	salario	especialidad	CodSuperv
1235	M. Franco	625000	Eléctrico	1311
1412	C. Nuñez	687500	Plomero	1520
2920	R. García	500000	Techos	2920
3231	P. Martínez	870000	Armado	3231
1520	H. Rivadeneira	587500	Plomero	1520
1311	C. Cortes	775000	Pisos	3231
3001	J. Barrientos	410000	Repello	3231

Obra

<u>IdObra</u>	direccion	tipo	comuna	estrato
312	Parque Versailles	Oficinas	1	5
435	Buga-	Carretera	0	1
515	Salida Jamundí	Residencias	19	6
210	San Fernando	Oficinas	19	4
111	Santa Mónica	Oficinas	1	6
460	Alameda	Almacén	19	2

Asignación

<u>codigo</u>	<u>idobra</u>	fechainic	nrodias
1235	312	38270	5
1412	312	37996	10
1235	515	38178	22
2920	460	38117	18
1412	460	38211	18
2920	435	47027	10
2920	210	38271	15
3231	111	38270	8
1412	435	42278	75
1412	515	38118	8
3231	312	45566	20
1520	515	38240	14
1311	435	38209	12
1412	210	42309	12

El Lenguaje de Consulta SQL

DEFINICION DE DATOS EN SQL

- a) Se suministra el nombre de la tabla.*
- b) Se define cada columna*
- c) Se asignan atributos de edición a cada columna*

Termino Relacional	Termino SQL
Tabla o Relación	Table
Fila o Tupla	Row
Columna o Atributo	Column
Esquema Relacional	Scheme

Crear: **CREATE**

Borrar: **DROP**

Modificar: **ALTER**

Creación de Tablas

DEFINICION DEL ESQUEMA

El esquema:

- Permite definir el conjunto de tablas y otros elementos de la base de datos
- Disponibles desde SQL2
- Identificables mediante un nombre
- La creación del esquema se puede hacer en un solo paso (especificando sus elementos). También se puede declarar el esquema y luego incorporarle sus elementos

Ejemplo:

CREATE SCHEMA CONSTRUCCION

Creación de Tablas

Definición de Datos, Tipos de Datos Y Restricciones

Creación de tablas: **CREATE TABLE**

Ejemplo

- i) `CREATE TABLE CONSTRUCCION.OBRA`
- ii) `CREATE TABLE OBRA`

Definición de una tabla

- Nombre de la tabla
- Declaración de atributos: nombre, dominio, restricciones
- Restricciones: clave primaria, integridad referencial y demás

Creación de Tablas

Una tabla en SQL se define en tres pasos:

- a) Se suministra el nombre de la tabla.*
- b) Se define cada columna*
- c) Se definen las llaves multicolumna y las llaves alternas.*

CREATE TABLE Empleado(
codigo

nombre
apellido
salario
especialidad
codSuperv int);
integer **NOT NULL PRIMARY KEY**,
varchar (30),
char (30)
numeric(12,3),
varchar (15),

CREATE TABLE Obra(
idObra

direccion varchar (20),
tipo varchar (12),
comuna smallint,
estrato smallint);
int **NOT NULL PRIMARY KEY**,

CREATE TABLE Asignacion(
codigo

idObra
fechaInic date,
nroDias smallint,
PRIMARY KEY (codigo, idObra));
int **NOT NULL**,
int **NOT NULL**,

Creación de Tablas

Forma general:

```
CREATE TABLE <Nombre de la tabla> (  
  <Atributo-1> <Tipo de dato> <[OPCIONES]>  
  <Atributo-2> <Tipo de dato> <[OPCIONES]>  
  ...  
  <Atributo-N> <Tipo de dato> <[OPCIONES]> );
```

<Tipo de dato>:

Datos numéricos

Enteros: INTEGER O INT, SMALLINT

Reales: FLOAT, REAL, DOUBLE PRECISION, DECIMAL(n, d)

Datos de Cadena

Cadena de Longitud Fija: CHARACTER (n) CHAR (n)

Cadena de Longitud Variable: VARCHAR (n)

Datos de Fecha/Hora

Fecha: DATE (año, mes, día – formato: aaaammdd)

Hora : TIME (hora, minutos, segundos – formato: hh:mm:ss)

<[Opciones]>: [NOT NULL], [UNIQUE], [FOREIGN KEY]

Creación y Borrado de índices

CREACION DE INDICES

Los índices nos ayudan a obtener datos de las tablas en forma más rápida. Ejemplo: estamos interesados en leer en un libro de jardinería acerca de cómo cultivar tomate. En vez de leer el libro desde el comienzo hasta que encontremos una sección sobre tomate, es mucho más rápido para nosotros ir al índice, ubicar qué páginas contienen la información sobre tomate, y luego dirigirnos a esas páginas directamente. Así ahorramos tiempo y seguramente es la forma más eficiente para ubicar la información buscada. Lo mismo se aplica para obtener datos en una tabla de una base de datos. Sin un índice, el sistema de base de datos lee a través de toda la tabla para localizar la información deseada. Con el índice correcto, el sistema de base de datos puede dirigirse al índice para encontrar de dónde obtener los datos, y luego dirigirse a dichas ubicaciones y obtener los datos necesarios.

Creación y Borrado de índices

CREACION DE INDICES

Por lo tanto, generalmente se recomienda crear índices en tablas. Un índice puede cubrir una o más columnas. La sintaxis general para la creación de un índice es:

Sintaxis:

```
CREATE INDEX <Nombre Índice> ON <Tabla-base>  
      (Columna-i <[Orden]>,..., Columna-j <[Orden]>);
```

donde : <[Orden]>: ASC o DESC

Ejemplo: **CREATE INDEX** IndEmp **ON** Empleado (código, ASC);

BORRADO DE INDICES

Sintaxis:

```
DROP INDEX <Nombre Índice>;
```

Ejemplo: **DROP INDEX** IndEmp;

Modificación y Borrado de Tablas

MODIFICACION DE LA ESTRUCTURA DE UNA TABLA

Una vez que se crea la tabla, hay ocasiones donde se necesita cambiar la estructura de la tabla, como: agregar un atributo, eliminar un atributo, cambiar el nombre de un atributo, cambiar el tipo de datos para un atributo.

Sintaxis:

ALTER TABLE <Tabla-base> **ADD** Atributo <tipo-de-dato>;

ALTER TABLE <Tabla-base> **DROP** Atributo;

ALTER TABLE <Tabla-base> **MODIFY** Atributo <Opción>

<OPCION>: NOT NULL, nuevo tamaño, nuevo Tipo de dato;

Ejemplo: (Añadir columna): **ALTER TABLE** Empleado **ADD** sexo CHAR (1);

Ejemplo (Eliminar columna): **ALTER TABLE** Empleado **DROP** sexo;

i) Propagación en cascada (CASCADE)

ii) No eliminar si hay relacionados (RESTRICT)

ALTER TABLE Cliente **DROP** ciudad **CASCADE**;

Modificación y Borrado de Tablas

BORRADO DE TABLAS

Eliminación de tablas: **DROP TABLE**

Eliminación de esquemas: **DROP SCHEMA**

Sintaxis:

DROP TABLE <TablaBase>;

Ejemplo: **DROP TABLE** OtrosEmpleados;

Si hay elementos relacionados se puede detener el proceso (RESTRICT) o se puede propagar el efecto (CASCADE)

Ejemplo:

DROP TABLE Cliente RESTRICT;

DROP SCHEMA Banco CASCADE;

Definir claves primarias y foráneas

Sintaxis:

Definición de clave: **PRIMARY KEY**

Los atributos que forman la clave deben ser NOT NULL

Definición de clave externa: FOREIGN KEY ... REFERENCES

Ejemplo:

```
CREATE TABLE Asignacion(  
codigo          int  NOT NULL,  
idObra          int  NOT NULL,  
fechaInic       date,  
nroDias         smallint,
```

```
PRIMARY KEY (codigo, idObra),
```

```
FOREIGN KEY (codigo) REFERENCES Empleado(codigo),
```

```
FOREIGN KEY (idObra) REFERENCES Obra(idObra)
```


Definir claves primarias y foráneas

Ejemplo 2:

```
CREATE TABLE Asignacion(  
codigo          int  NOT NULL,  
idObra          int  NOT NULL,  
fechaInic       date,  
nroDias          smallint);
```

```
ALTER TABLE Asignacion ADD CONSTRAINT  
PK_Asignacion PRIMARY KEY (codigo, idObra);
```

```
ALTER TABLE Asignacion ADD CONSTRAINT  
PK_Asignacion_Empleado FOREIGN  
KEY(codigo) REFERENCES Empleado(codigo);
```

```
ALTER TABLE Asignacion ADD CONSTRAINT  
PK_Asignacion_Obra FOREIGN KEY (idObra) REFERENCES  
Empleado(idObra);
```

Trabajo con Oracle o MariaDB

Oracle:

<https://www.oracle.com/es/database/technologies/appdev/xe.html>

MariaDB:

https://mariadb.org/download/?t=mariadb&p=mariadb&r=10.11.5&os=windows&cpu=x86_64&pkg=msi&m=insacom

Consulta de Datos

El SQL provee una gran variedad de capacidades de manipulación de datos para actualización y consultas a la base de datos, las que dependen solo de la estructura lógica de la base de datos, no de su estructura física. El lenguaje SQL es **insensible a mayúsculas y minúsculas**, es decir, las trata por igual. Por tanto, las palabras clave **SELECT**, **FROM**, **WHERE** es igualmente correcto escribirlas como **select**, **from**, **where** o **Select**, **From**, **Where**. Igual tratamiento reciben los nombres de atributos.

CONSULTAS SIMPLES

Una consulta simple es aquella que involucra solamente una tabla de la base de datos.

Consulta: ¿Quiénes son plomeros?

SELECT nombre

FROM Empleados

WHERE especialidad = "Plomero";

Output pane	
Data Output	Explain Messages
	nombre character varying(30)
1	C. Nuñez
2	H. Rivadeneira

Funcionamiento del SQL

El query ilustra las tres cláusulas del SQL más utilizadas: **SELECT**, **FROM** y **WHERE**.

Cláusula SELECT.

La cláusula **SELECT** lista las columnas deseadas en el resultado de la consulta, las cuales siempre serán las columnas de una tabla relacional. También se puede incluir varias columnas o incluir valores calculados y valores literales.

Cláusula FROM.

La cláusula **FROM** lista una o más tablas que son referenciadas en la consulta. Todas las columnas listadas en las cláusulas **SELECT** o **WHERE** deben hallarse en una de las tablas de la cláusula **FROM**.

Cláusula WHERE.

La cláusula **WHERE** contiene una condición para seleccionar filas de las tablas listadas en la cláusula **FROM**. Es la cláusula más versátil del SQL y puede contener una gran variedad de condiciones.

Funcionamiento del SQL

La consulta SQL es procesada por el sistema, en el orden **FROM**, **WHERE**, **SELECT**. Es decir, las filas de la tabla referenciada en la cláusula **FROM** son ubicadas en un área de trabajo para el procesamiento, realizándose el producto cartesiano de las tablas listadas.

Luego la cláusula **WHERE** es aplicada a cada fila, una por una. Cualquier fila que no satisface la cláusula **WHERE** es eliminada de este grupo, las filas que sí satisfacen el **WHERE** son luego procesadas por la cláusula **SELECT**. Ejemplo:

CONSULTA: Liste la información acerca de oficinas en construcción.

```
SELECT *  
FROM Obras  
WHERE tipo = "Oficinas";
```

Output pane					
Data Output Explain Messages History					
	idobra integer	direccion character varying(20)	tipo character varying(12)	comuna smallint	estrato smallint
1	312	Parque Versailles	Oficinas	1	5
2	210	San Fernando	Oficinas	19	4
3	111	Santa Mónica	Oficinas	1	6

Forma General de la Consulta

El verbo **SELECT** que define todas las operaciones de consulta, es el más importante del lenguaje.

La forma o sintaxis general de una operación **SELECT** es la siguiente:

```
SELECT [distinct] [*] |<lista de atributos>  
FROM tabla(s)  
[WHERE condición(es)]  
[group by <atributo(s)> having condición(es)]  
[order by <atributo(s)>];
```

Los corchetes indican construcciones opcionales. El verbo **SELECT** no corresponde con la operación de selección del álgebra relacional.

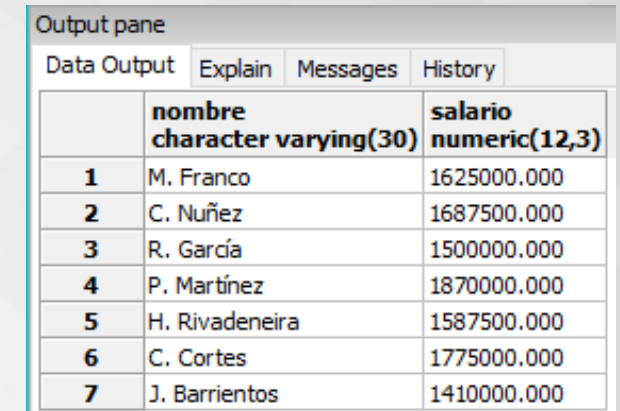
¡Es mucho más poderosa!

Proyecciones usando SQL

Para formar una proyección (π) en SQL nombramos la relación a proyectarse y listamos las columnas que van a ser mostradas.

Ejemplo: Listar el nombre y el salario de cada empleado.

SELECT nombre, salario
FROM Empleados;



Output pane

	nombre character varying(30)	salario numeric(12,3)
1	M. Franco	1625000.000
2	C. Nuñez	1687500.000
3	R. García	1500000.000
4	P. Martínez	1870000.000
5	H. Rivadeneira	1587500.000
6	C. Cortes	1775000.000
7	J. Barrientos	1410000.000

Las palabras reservadas **SELECT** y **FROM** siempre se requieren; las columnas que deben presentarse (nombre, salario) se listan después de la palabra reservada **SELECT**; y la tabla a utilizar (Empleado) se define después de la cláusula **FROM**.

No se debe confundir la cláusula **SELECT**, con el operador de selección del álgebra relacional (σ). El verbo **SELECT** se usa para realizar la proyección del Álgebra Relacional.

Proyecciones usando SQL

Consideremos el siguiente ejemplo:

SELECT especialidad
FROM Empleados;

	especialidad character varying(15)
1	Eléctrico
2	Plomero
3	Techos
4	Armado
5	Plomero
6	Pisos
7	Repello

Como puede verse, la tabla contiene **renglones duplicados**. Por lo tanto, en sentido estricto esta tabla **no es una relación**. SQL no elimina automáticamente los duplicados. Si se desea eliminar las tuplas repetidas, se debe especificar el calificador **distinct**:

SELECT DISTINCT especialidad
FROM Empleados;

Consulta que lista las especialidades de los empleados.

	especialidad character varying(15)
1	Techos
2	Eléctrico
3	Armado
4	Pisos
5	Plomero
6	Repello

Selecciones usando SQL

El operador de selección del álgebra relacional (σ) También se realiza en SQL con la cláusula **WHERE**:

Ejemplo: Listar las obras del tipo “oficinas”

SELECT *
FROM Obras
WHERE tipo = “Oficinas”;

	idobra integer	direccion character varying(20)	tipo character varying(12)	comuna smallint	estrato smallint
1	312	Parque Versalles	Oficinas	1	5
2	210	San Fernando	Oficinas	19	4
3	111	Santa Mónica	Oficinas	1	6

Esta expresión, especifica los nombres de **todas** las columnas de las tablas (**mediante el uso del ***). La cláusula **FROM** especifica la tabla a utilizar (Obras). Y la cláusula **WHERE** suministra las condiciones inherentes a la selección.

Se pueden expresar varias condiciones en la cláusula **WHERE**.

También podemos combinar las cláusulas **SELECT** y **WHERE** (para realizar la combinación de las operaciones de **proyección y de selección del álgebra relacional**).

Cláusula WHERE

La cláusula **WHERE** sirve para restringir mediante una fórmula, el conjunto de tuplas que se desea extraer.

Ejemplo: ¿Cuál es el salario promedio diario de cada electricista?

SELECT nombre, "Salario Promedio Diario = ", salario/30
FROM Empleados
WHERE especialidad = "Eléctrico"
ORDER BY nombre;

	nombre character varying(30)	?column? unknown	?column? numeric
1	M. Franco	Salario Prom	54166.666€

En el ejemplo se muestra el uso de literales alfanuméricos "Salario Promedio Diario = " y cálculos dentro de la cláusula **SELECT** salario/30.

Se ha incluido una nueva cláusula (**ORDER BY**) que se usa para clasificar el resultado de la consulta en orden ascendente o descendente por una columna deseada (nombre)

Cláusula WHERE

En la cláusula **WHERE** se pueden usar condiciones complejas formadas con los operadores **and**, **or** y **not** y los comparadores **=**, **<**, **>**, **<=**, **>=**. También se incluyen **between** y **not between**:

atributo between valor1 and valor2

*donde valor1 **<=** valor2 y la condición será verdadera si*

***atributo** pertenece al rango [valor1, valor2]*

*Otros predicados son **null** y **like**. **null** se usa para determinar si un componente de una tupla es nulo, su forma es:*

atributo [not] is null

*El predicado **like** se utiliza para la búsqueda de patrones con los datos. Se usan los caracteres especiales % y _, su forma es:*

atributo like “patrón de caracteres”

Cláusula WHERE

Consulta: ¿Quienes ganan entre \$1.550.000 y \$1.750.000?

SELECT *

FROM Empleados

WHERE salario **between** 1550000 **and** 1750000;

	codigo integer	nombre character varying(30)	salario numeric(12,3)	especialidad character varying(15)	codsuperv integer
1	1235	M. Franco	1625000.000	Eléctrico	1311
2	1412	C. Nuñez	1687500.000	Plomero	1520
3	1520	H. Rivadeneira	1587500.000	Plomero	1520

Consulta: ¿Empleados con especialidad plomero, techos, pisos?

SELECT *

FROM Empleados

WHERE especialidad **in** ("plomero", "techos", "pisos");

	codigo integer	nombre character varying(30)	salario numeric(12,3)	especialidad character varying(15)	codsuperv integer
1	1412	C. Nuñez	1687500.000	Plomero	1520
2	2920	R. García	1500000.000	Techos	2920
3	1520	H. Rivadeneira	1587500.000	Plomero	1520
4	1311	C. Cortes	1775000.000	Pisos	3231

Consulta a varias tablas

La habilidad de conectar datos a través de varias tablas es esencial en un lenguaje de Bases de Datos. En Álgebra relacional, esto se logra con el operador **JOIN**.

Para realizar una junta, simplemente listamos más de una relación en la cláusula **FROM** indicando en la cláusula **WHERE** las condiciones de junta.

Consulta: ¿Cuáles son las especialidades de los empleados asignados a la obra 435?

```
SELECT especialidad  
FROM Empleados, Asignaciones  
WHERE Empleados.codigo = Asignaciones.codigo and  
idObra = 435;
```

	especialidad character varying(15)
1	Plomero
2	Techos
3	Pisos

Los datos de la consulta se encuentran en las tablas **Empleados** y **Asignaciones**.

Consulta a varias tablas

¿Qué sucede?

Como es usual, la cláusula **FROM** es procesada primero, aunque, ya que hay **dos tablas involucradas**, el sistema crea primero el **PRODUCTO CARTESIANO** de las filas de esas tablas. Es decir, que se crea una gran tabla que contiene todas las columnas de ambas tablas. Si hay más de dos tablas en la cláusula **FROM**, el **producto cartesiano** es creado desde todas las tablas.

Luego se aplica la cláusula **WHERE**. Cada fila de la relación creada por la cláusula **FROM** es examinada de acuerdo con la cláusula **WHERE**. Las tuplas que NO satisfacen la condición **WHERE** se eliminan. Luego se aplica la cláusula **SELECT** a las filas resultantes. Para definir cual código se utiliza, se cualifica su relación: **Empleados.codigo** y **Asignaciones.codigo**

Consulta a varias tablas

Veamos cómo se puede unir una relación con ella misma en SQL:
Consulta: Liste los trabajadores con el nombre su supervisor.

```
SELECT A.nombre, B.nombre  
FROM Empleados AS A, Empleados AS B  
WHERE A.codSuperv = B.codigo;
```

	nombre character varying(30)	nombre character varying(30)
1	M. Franco	C. Cortes
2	C. Nuñez	H. Rivadeneira
3	R. García	R. García
4	P. Martínez	P. Martínez
5	H. Rivadeneira	H. Rivadeneira
6	C. Cortes	P. Martínez
7	J. Barrientos	P. Martínez

La cláusula **FROM** en este ejemplo define dos copias de la relación Empleado, dándole nombres alias **A** y **B**. Las copias **A** y **B** de Empleado son unidas en la cláusula **WHERE** colocando el **codigo** en **B** igual a **codSuperv** en **A**. **A.nombre** representa el empleado y **B.nombre** representa el supervisor. En SQL se pueden unir más de dos relaciones al mismo tiempo.

Consulta a varias tablas

Consulta: Liste los nombres de los trabajadores asignados a obras oficinas.

```
SELECT nombre  
FROM Empleados, Asignaciones, Obras  
WHERE Empleados.codigo = Asignaciones.codigo and  
       Asignaciones.idObra = Obras.idObra and  
       tipo = "oficinas";
```

Resultado:

	nombre character varying(30)
1	M. Franco
2	C. Nuñez
3	R. García
4	P. Martínez
5	P. Martínez
6	C. Nuñez
7	C. Nuñez
8	J. Barrientos
9	H. Rivadeneira

Subconsultas

Una subconsulta o “consulta dentro de una consulta” puede ser colocada dentro de una cláusula **WHERE** de una consulta, resultando en una expansión de las capacidades de esta cláusula.

Consulta: ¿Cuáles son las especialidades de los empleados asignados a la obra 435?

```
SELECT especialidad  
FROM Empleados  
WHERE codigo in  
      (SELECT codigo  
       FROM Asignaciones  
       WHERE idObra = 435);
```

	especialidad character varying(15)
1	Techos
2	Plomero
3	Pisos

La subconsulta del ejemplo es :

```
(SELECT codigo  
 FROM Asignaciones  
 WHERE idObra = 435);
```

	codigo integer
1	2920
2	1412
3	1311

Subconsultas

El resultado de la subconsulta se convierte en un conjunto que se usará al ejecutar la consulta principal. Esta consulta causa que cada fila de empleado sea evaluada con respecto al contenido de tal conjunto en la cláusula **WHERE**. Si la fila correspondiente al **codigo** (de **Empleados**) está en (**in**) el conjunto generado por la subconsulta, entonces la **especialidad** de la fila que SI está trabajando es seleccionada y mostrada en el resultado

Es muy importante que la cláusula **SELECT** de la subconsulta contenga el **codigo**, solamente. De otra forma, la cláusula **WHERE** de la consulta principal, que pregunta si el **codigo** **está en el conjunto** de **Empleados** (**códigos**), no tendrá respuesta.

La semántica de **in** es la pertenencia a un conjunto. La subconsulta **debe producir una relación unaria**, o sea un **conjunto** y la condición es cierta para una tupla dada **si el valor del atributo en esa tupla pertenece al conjunto** producto de la subconsulta.

Subconsultas

Ejemplo: Liste los nombres de los trabajadores asignados a las obras oficinas.

```
SELECT nombre
FROM Empleados
WHERE codigo in
    (SELECT codigo
     FROM Asignaciones
     WHERE idObra in
        (SELECT idObra
         FROM Obras
         WHERE tipo = "Oficinas"));
```

	nombre character varying(30)
1	M. Franco
2	C. Nuñez
3	R. García
4	P. Martínez
5	H. Rivadeneira
6	J. Barrientos

En estas consultas, las subconsultas **más internas** son evaluadas **primero**.

Renombrar Atributos

Para renombrar atributos se usa la palabra **AS**

SELECT nombre **AS** nombreEmpleado
FROM Empleados;

	nombreempleado character varying(30)
1	M. Franco
2	C. Nuñez
3	R. García
4	P. Martínez
5	H. Rivadeneira
6	C. Cortes
7	J. Barrientos

Es bastante útil cuando se tienen nombres “complicados” o largos para una columna o se han realizado operaciones.

Ejemplo:

SELECT codigo, salario*1.05 **AS** NewSal
FROM Empleados;

Resultado:

	codigo integer	newsal numeric
1	1235	1706250.00
2	1412	1771875.00
3	2920	1575000.00
4	3231	1963500.00
5	1520	1666875.00
6	1311	1863750.00
7	3001	1480500.00

Subconsultas Correlacionadas

Esta clase de subconsultas son aquellas cuyos valores por ejecución **dependen** de la fila que está siendo examinada en la consulta principal.

CONSULTA: Liste los empleados que reciben un mayor salario promedio diario que sus supervisores.

Es decir, que la fila del supervisor a examinar *depende directamente* de la fila del empleado que está siendo examinado.

```
SELECT A.nombre  
FROM Empleados A  
WHERE A.salario/30 >  
      (SELECT B.salario/30  
       FROM Empleados B  
       WHERE A.codSuperv = B.codigo);
```

	nombre character varying(30)
1	C. Nuñez

Subconsultas Correlacionadas

Los siguientes son los pasos en la ejecución de la consulta:

1. El sistema hace dos copias de la relación Empleados (**A** y **B**). **A** se refiere al empleado y **B** al supervisor.
2. El sistema luego examina cada fila de **A**. Una fila de **A** es seleccionada si satisface la condición en la cláusula **WHERE** (**salario** es **mayor** que el salario generado por la subconsulta).
3. La subconsulta selecciona el salario diario de las filas de **B** cuyo **codigo** es **igual** al **codSuperv** de la fila de **A** que se está examinando en la consulta principal. Este es el **salario diario** del supervisor de **A**.

La subconsulta debe generar solo un valor. Este valor cambia dependiendo de la fila de **A** que está siendo examinada.

Consultas Anidadas

Una tercera forma de especificar subconsultas es mediante el uso del cuantificador universal **exists**.

CONSULTA: identificar todos los obreros no asignados a la obra 435.

```
SELECT codigo  
FROM Asignaciones  
WHERE idObra <> 435;
```

Infortunadamente, esta consulta solamente nos da los códigos de los trabajadores que están trabajando en otras obras distintas a la 435. La solución correcta debe usar el operador **not exists**:

```
SELECT codigo  
FROM Empleados  
WHERE not exists  
    (SELECT *  
     FROM Asignaciones  
     WHERE idObra = 435 and  
           Asignaciones.codigo = Empleados.codigo);
```

	codigo integer
1	1235
2	1412
3	1235
4	2920
5	1412
6	2920
7	3231
8	1412
9	3231

	codigo integer
1	1235
2	3231
3	1520
4	3001

Consultas Anidadas

Los operadores **exists** y **not exists** siempre preceden la subconsulta.

El **exists** evalúa como “verdadero” si el conjunto resultante de la subconsulta **no es vacío**, de lo contrario lo evalúa como “falso”. El operador **not exists**, trabaja de una forma opuesta.

¿Por qué se debería usar exists o not exists?

La opción **exists** provee el único medio disponible para resolver consultas que contengan el cuantificador “**TODOS**” en la condición. Anteriormente, en el tema del álgebra relacional, tales consultas se resolvían con la división. Ejemplo:

CONSULTA: Liste los empleados que están asignados a todas las obras.

Consultas Anidadas

Esta consulta se puede resolver usando una **doble negación**. A continuación replantearemos la consulta:

CONSULTA: Liste los empleados tales que **NO** haya construcción en la cual **NO** estén asignados.

Veamos antes, una solución a otro problema: Identificar las obras en las cuales un trabajador dado (el **1234**) **NO** es asignado:

```
SELECT idObra
FROM Obras
WHERE not exists
    (SELECT *
     FROM Asignaciones
     WHERE Asignaciones.Codigo = 1234 and
           Asignaciones.idObra = Obras.idObra);
```

Predicados de Comparac. Cuantificados

Estos predicados son θ , *some*, *any* y *all*, donde θ es un operador de comparación aritmético. *some*, *any* representan al cuantificador existencial y *all* al cuantificador universal. El operador θ acompaña a los cuantificadores anteriores vinculando una expresión con una subconsulta:

Expresión [*all* | *some* | *any*] (*subconsulta*)

El predicado θ_any o θ_some se satisface si existe al menos una fila del resultado de la subconsulta que está en relación θ con *Expresión*.

El predicado θ_all se satisface si toda tupla de la subconsulta está en relación θ con *Expresión*.

CONSULTA: identificar todos los obreros que están asignados a obras “Carretera”, “Residencias” o “Almacén”.

Predicados de Comparac. Cuantificados

```
SELECT nombre
FROM Empleado
WHERE codigo in
  (SELECT código
   FROM Asignacion
   WHERE idObra = some
     (SELECT idObra
      FROM Obra
      WHERE tipo in
        ("Carretera", 'Residencias', "Almacen")))) ;
```

	nombre character varying(30)
1	M. Franco
2	C. Nuñez
3	R. García
4	H. Rivadeneira
5	C. Cortes

CONSULTA: nombre del trabajador que devenga el menor salario.

```
SELECT nombre FROM Empleado
WHERE salario <= all (SELECT salario FROM Empleado) ;
```

	nombre character varying(30)
1	J. Barrientos

Condiciones en que intervienen Relac.

Hay varios operadores SQL que aplican a una relación R produciendo un resultado booleano (**in**, **all**, **any**) y requieren de un escalar s :

1. **exists** R da verdadero si y solo si R no es vacío.
2. s **in** R es verdadero si y solo si s es igual a uno de los valores de R . s **not in** R es verdadero si y solo si s no es igual a ningun valor de R .
3. s **> all** R es verdadero si y solo si s es mayor que cualquier valor de la relación unaria R . El operador **>** puede ser reemplazado por cualquier otro de los operadores de relación. s **<> all** R es lo mismo que s **not in** R .
4. s **> any** R es verdadero si y solo si s es mayor que al menos un valor de la relación unaria R . cualquier otro de los operadores de relación podrá reemplazar el **>**. s **= any** R es lo mismo que s **in** R .

Variable de Tupla y nombre de Relac.

Para lograr eliminar ambigüedades en los atributos se debe **anteponer** el **nombre de la relación** siempre y cuando en la consulta se combinen varias relaciones.

Cuando en una consulta se necesita manejar dos o más tuplas procedentes de una misma relación **R**, **la podemos incluir en la cláusula FROM cuantas veces se necesite, cada ocurrencia de R se define con su propia variable de tupla (“alias”)**.

Desde el punto de vista técnico, las referencias a atributos en las cláusulas **SELECT** y **WHERE** **siempre se refieren a una variable de tupla** (alias).

Pero si una relación aparece sólo una vez en la cláusula **FROM**, podemos utilizar el nombre de ella como su propia variable de tupla. Así el nombre de una relación **R** en la cláusula **FROM** puede considerarse una abreviatura de **R AS R**.

Comparación de Cadenas

En cualquier momento, se puede necesitar información sobre las filas que tengan valores similares en una columna determinada. Es decir, se requiere seleccionar filas basadas en algún modelo. En SQL se usa la palabra reservada **like** en la cláusula **WHERE** (en lugar del operador de comparación) para la búsqueda de patrones. No se requiere conocer el valor de la columna completa para identificar la fila y escogerla.

El formato es:

WHERE < atributo> **like** modelo

La coincidencia de modelos solo aplica a columna de cadenas.

SQL debe proporcionar alguna idea de donde está la cadena de caracteres parcial. Se usan dos caracteres especiales a modo de comodín, que forman parte del modelo. Los caracteres son **%** (porcentaje) y **_** (subrayado).

Comparación de Cadenas

El (%) se interpreta como un carácter que puede representar cualquier cadena de caracteres de cualquier longitud (hasta de tamaño cero).

Ejemplo: Listar la identidad, tipo y dirección de Obras cuya dirección inicia por Sa (San Fernando, Santa Mónica, Salida Jamundí, ...)

```
SELECT idObra, tipo, dirección  
FROM Obras  
WHERE direccion like 'Sa%';
```

	idobra integer	direccion character varying(20)	tipo character varying(12)	comuna smallint	estrato smallint
1	515	Salida Jamundí	Residencias	19	6
2	210	San Fernando	Oficinas	19	4
3	111	Santa Mónica	Oficinas	1	6

El (__) permite comparar un solo carácter en que aparezca el símbolo _ .

Ejemplo: Listar el nombre del empleado cuyo supervisor tenga un código tenga los 3 en el 1er carácter y 3 en el 3er carácter.

Comparación de Cadenas

```
SELECT nombre  
FROM Empleado  
WHERE codSuperv LIKE '3_3_';
```

ERROR: el operador no existe: integer ~~ unknown

LINE 3: WHERE codSuperv LIKE '3_3_';
 ^

HINT: Ningún operador coincide con el nombre y el tipo de los argumentos. Puede desear agregar conversiones explícitas de tipos.

***** Error *****

El operador **LIKE** solo compara cadenas de caracteres. Por lo tanto, necesitamos realizar un casting explícito de la columna entera a una cadena de caracteres (string) como se muestra a continuación.

```
SELECT nombre  
FROM Empleado  
WHERE TO_CHAR(codSuperv) LIKE '3_3_';
```

	nombre character varying(30)
1	P. Martínez
2	C. Cortes
3	J. Barrientos

Comparación de Fecha y Hora

Los valores de fecha pueden representarse en varios formatos como 04/12/2014 o 12 Abril 2014.

La fecha se representa con la palabra **DATE** seguida de una cadena entre comillas simples de una forma especial. Ejemplo: **DATE** '2004-04-12' tiene la forma requerida y representa año-mes-día.

Una hora se representa con la palabra **TIME** y una cadena entre comillas. La cadena tiene dos dígitos para la hora (en la hora militar de 24 horas). Luego vienen dos puntos, dos dígitos para los minutos, dos puntos y dos dígitos para los segundos un punto y un dígito para las décimas de segundo. **TIME** '18:00:02.5' representa la hora de inicio del curso Bases de Datos I, el día martes.

Se pueden comparar fechas u horas usando los operadores de comparación conocidos.

Unión, Intersección y Diferencia

A veces se quiere combinar relaciones usando las operaciones del álgebra relacional: unión, intersección y diferencia. SQL ofrece los operadores **UNION**, **INTERSECT** y **EXCEPT**.

Las palabras clave (operadores) se emplean entre dos consultas, que deben ir entre paréntesis. Ejemplo, sean las relaciones:

EstrellasPelicula (nombre, direccion, sexo, fechaNacmto)

EjecutivosCine (nombre, direccion, codProductor, ingresoNeto)

**(SELECT nombre, direccion FROM EstrellasPelicula
WHERE sexo = 'F')**

INTERSECT

**(SELECT nombre, direccion FROM EjecutivosCine
WHERE ingresoNeto > 10000000);**

Se quiere el nombre y dirección de las estrellas de sexo femenino que además sean ejecutivas cinematográficas con un capital neto superior a \$10.000.000 de dólares.

Unión, Intersección y Diferencia

CONSULTA: Se requieren los nombres y direcciones de las estrellas del cine que NO son ejecutivos cinematográficos:

```
(SELECT nombre, direccion FROM EstrellasPelicula )
```

EXCEPT

```
(SELECT nombre, direccion FROM EjecutivosCine );
```

CONSULTA: Suponga que queremos los títulos y año de las películas que figuran en la lista de Películas y en EstrellaPelícula

EstrellaPelícula (tituloPelicula, añoPelicula, nombreEstrella)

Película (titulo, año, aColor, estudio, productor)

```
(SELECT titulo, año FROM Pelicula )
```

UNION

```
(SELECT tituloPelicula as titulo, añoPelicula as año  
FROM EstrellaPelicula );
```

Eliminación de Duplicados

SQL maneja un concepto de relación que difiere de la noción abstracta presentada inicialmente, ya que utiliza relaciones que son **bolsa** y **no conjuntos**, por tanto, una tupla **puede aparecer más de una vez** dentro de una relación.

Nada impide que SQL posea **duplicados**, si las relaciones originales poseen duplicados el producto cartesiano tendrá duplicados y las copias idénticas se parean, incrementando la duplicación.

Si no se quieren duplicados se debe utilizar la palabra clave **DISTINCT**, después de la cláusula **SELECT**

Las operaciones de **unión**, **intersección** y **diferencia** normalmente los suprimen. A fin de evitar que ello suceda, es necesario agregar enseguida de los operadores **UNION**, **INTERSECT** o **EXCEPT**, la palabra clave **ALL**.

Costo de Eliminar Duplicados

Se puede tener la tentación de colocar la palabra clave **DISTINCT** después de toda cláusula **SELECT**, por suponer que no se produce ningún daño.

La realidad es que resulta **bastante costoso** eliminar duplicados de una relación: hay que **ordenar la relación** de modo que las tuplas idénticas aparezcan **contiguas** (solo agrupándolas se puede determinar si una tupla debe eliminarse o no).

El **tiempo** que dura el ordenamiento de las relaciones a menudo **mayor** que el que tarda la ejecución de la consulta.

En conclusión, **hay que utilizar la eliminación con mucha prudencia**, si se quiere ejecutar la consulta **rápidamente**.

